

Traffic Congestion Detection and Classification with Sensor-Noise Robustness: A Digital Twin-Enabled Multi-Model Machine Learning Approach

Assem Mohamed Saad¹, Omar Ahmed Asaad¹, Habiba Hosam Eldin¹

Seif Adel Elbasha¹, Owida Elsayed¹

{200030013, 200027710, 200025297, 200046355, 200042768}@must.edu.eg

Dr. Amr Ibrahim¹, Dr. Walid Hamdy¹

{amr.ibrahim, walid.hamdy}@must.edu.eg

¹Faculty of Information Technology, Misr University for Science and Technology (MUST), Giza, Egypt

Abstract

Urban traffic congestion is one of the most persistent challenges facing modern smart cities, imposing significant economic, environmental, and social costs. Despite growing adoption of data-driven approaches for traffic monitoring, most existing systems treat sensor data as noise-free and reliable—an assumption that fails in real-world deployments where cameras suffer from occlusion and sensors experience dropout events. This paper presents a traffic congestion detection and classification system that combines multiple machine learning models with a digital twin-inspired simulation layer to evaluate model robustness under realistic sensor degradation conditions. Using a two-month dataset of vehicle counts sampled at 15-minute intervals, we train and compare five classifiers—Logistic Regression, Random Forest, Extra Trees, Histogram Gradient Boosting, and a fully-connected Neural Network—on a four-class congestion labeling task (normal, low, high, heavy). Beyond standard clean-data evaluation, we systematically inject simulated occlusion noise (± 3 vehicle count error) and sensor dropout (2% of readings) to benchmark each model’s resilience. Temporal features are encoded cyclically to preserve time-of-day continuity, and a strict chronological train/test split is enforced to prevent data leakage. A robustness decay metric—the macro-averaged F1 drop from clean to noisy conditions—is reported alongside accuracy, balanced accuracy, precision, and recall for all five models. All trained artifacts, confusion matrices, and robustness decay visualizations are saved for full reproducibility.

Index Terms

traffic congestion classification, sensor noise robustness, digital twin simulation, machine learning, ensemble methods, neural networks, intelligent transportation systems

I. INTRODUCTION

The rapid growth of urban populations has intensified the demand for intelligent transportation systems (ITS) capable of monitoring, classifying, and responding to traffic conditions in real time. Traffic congestion not only wastes fuel and increases travel times, but also contributes disproportionately to carbon

emissions and urban air pollution [1]. As cities invest in sensor infrastructure—loop detectors, camera-based vehicle counters, acoustic sensors, and GPS probes—the volume of available traffic data has grown substantially, creating an opportunity for machine learning-based congestion classification systems.

However, translating this data richness into reliable real-world systems remains non-trivial. Sensor deployments are inherently imperfect: cameras are occluded by large vehicles or adverse weather, sensor connections drop intermittently, and count readings are subject to systematic biases [9], [14]. The majority of published ML systems for traffic classification evaluate only on clean test sets, ignoring the performance degradation that occurs once deployed in the field. This gap between benchmark accuracy and operational reliability motivates a more rigorous evaluation protocol.

Parallel to this, the concept of the digital twin—a virtual replica of a physical system that mirrors its real-time state—has gained traction as a framework for testing and validating decisions before they are applied to real infrastructure [4], [7]. By simulating sensor failure scenarios within a digital twin environment, we can stress-test classification models without waiting for failures to occur in production.

This paper addresses both challenges. We build a multi-model traffic congestion classification system trained on two months of real sensor data and evaluate each model under both clean and synthetically degraded conditions. The degradation simulation mirrors known failure modes: random count perturbations (occlusion) and full zero-readings (sensor dropout). We compare five classifiers spanning traditional, ensemble, and neural network families, and report a robustness decay metric—the F1-Macro drop from clean to noisy conditions—as a primary evaluation criterion alongside standard accuracy metrics.

The remainder of this paper is organized as follows. Section II formally states the problem. Section III describes our contributions relative to existing work. Section IV reviews related literature. Section V details our proposed methodology. Section VI presents results and discussion. Section VII concludes the paper and outlines future directions.

II. PROBLEM STATEMENT

Let the feature vector at time step t be

$$\mathbf{x}_t = [\text{CC}, \text{BC}, \text{BuC}, \text{TC}, \text{HS}, \text{HC}, \mathbf{d}],$$

where CC, BC, BuC, and TC are vehicle counts (car, bike, bus, and truck), HS and HC are cyclic hour projections, and $\mathbf{d}$ is a one-hot encoding of the day of the week. The classification target $y_t \in \{\text{normal}, \text{low}, \text{high}, \text{heavy}\}$ represents the congestion level observed at that time step.

The dataset $\mathcal{D} = \{(\mathbf{x}_t, y_t)\}_{t=1}^T$ consists of observations sampled at 15-minute intervals over a two-month period, yielding $T \approx 5,952$ records and 96 samples per day. The class distribution is imbalanced, with *normal* comprising approximately 61% of records and *high* only 6%.

Two evaluation regimes are defined:

- **Clean regime:** Models are evaluated on the held-out test set $\mathcal{D}_{\text{test}}$ with original sensor readings.
- **Noisy regime:** A perturbed test set $\tilde{\mathcal{D}}_{\text{test}}$ is constructed by applying additive uniform noise $\epsilon \sim \mathcal{U}(-3, 3)$ to each vehicle count and randomly zeroing 2% of count readings to simulate dropout.

The core challenge is to learn a classifier $f: \mathbf{x} \mapsto y$ that achieves high F1-Macro on $\mathcal{D}_{\text{test}}$ while minimizing the *robustness decay*:

$$\Delta F1 = F1_{\text{macro}}^{\text{clean}} - F1_{\text{macro}}^{\text{noisy}} \quad (1)$$

A system that scores well on clean data but degrades sharply under noise is unsuitable for real-world deployment. A secondary challenge is the time-series nature of the data: random splitting would leak future information into training. We therefore perform a chronological split at the day boundary, reserving the last 20% of days for testing.

III. CONTRIBUTIONS

Relative to the existing literature reviewed in Section IV, this work contributes the following:

1. Robustness-first evaluation protocol. Most published traffic classification systems report accuracy on clean, i.i.d. test sets [8], [10], [12]. We introduce a paired evaluation—clean vs. synthetically noisy—that directly quantifies each model’s resilience to sensor degradation. This protocol is inspired by digital twin simulation principles [1], [7] and is absent from the majority of ML-for-traffic benchmarks.

2. Simulation of physically grounded sensor failure modes. Rather than applying generic Gaussian noise, our perturbation model specifically targets occlusion (± 3 count error per vehicle type) and sensor dropout (2% zero-out rate)—failure modes documented in real sensor deployments [9], [14]. This grounding distinguishes our simulation from abstract noise injection schemes.

3. Multi-family model comparison on a real-world tabular traffic dataset. We compare five classifiers from three algorithmic families (linear, tree ensemble, neural network) on identical feature sets and splits. While general tabular benchmarks exist [11], comparative traffic-specific benchmarks using balanced accuracy and macro-averaged metrics alongside robustness measures are sparse [12], [13].

4. Temporally faithful feature engineering and train/test splitting. We encode time-of-day as sine/cosine projections to preserve cyclic continuity and enforce a strict chronological split at the day level—practices not consistently adopted in traffic ML literature [5], [8].

5. Reproducible artifact pipeline. All trained models, scalars, label encoders, confusion matrices, and robustness decay plots are saved programmatically, enabling direct reproduction and deployment.

IV. RELATED WORK

A. Traffic Congestion Detection and Classification

Traffic flow modeling has evolved from classical parametric methods toward data-driven approaches. A comprehensive survey by Kashyap et al. [8] categorizes the field into *prediction* (estimating future flow values) and *classification* (assigning discrete congestion labels) tasks, noting that classification is better suited to real-time signal control and alert systems. Our work falls squarely in the classification category, targeting a four-level taxonomy also adopted in recent IEEE and Springer studies [10], [12].

Inputs to traffic classification systems vary considerably across the literature. Image-based systems use CNN and YOLO architectures to detect vehicles directly from camera feeds [5], while sensor-based systems operate on pre-aggregated count or speed data [10]. Mahmoud et al. [10] train classifiers on

sensor-derived vehicle counts—the same input modality as our system—and report that Random Forest consistently outperforms SVM and Logistic Regression baselines. A comparative study [12] confirms that ensemble methods are competitive with deep sequence models (LSTM, GRU) on structured tabular traffic data, particularly when training sets are limited in size. Our dataset uses vehicle counts across four categories (car, bike, bus, truck) at 15-minute granularity, closely mirroring the input structure of IoV-based monitoring systems [2]. Class imbalance is addressed through class-weighted objectives and balanced accuracy reporting.

B. ML Model Comparison: Ensembles vs. Neural Networks

A long-standing open question in applied ML is when neural networks outperform gradient-boosted decision trees on tabular data. McElfresh et al. [11] benchmark 19 algorithms across 176 datasets at NeurIPS 2023, finding that the NN vs. GBDT performance gap is frequently negligible and that light GBDT hyperparameter tuning often matters more than model choice—directly motivating our head-to-head comparison on traffic data.

Ting et al. [13] compare Random Forest against multiple graph convolutional network (GCN) variants for traffic prediction, finding that RF remains a strong baseline even against spatially-aware deep models. In our setting, which lacks graph structure (no road network topology), tree ensembles are expected to be especially competitive. We extend the comparison to Extra Trees, Histogram Gradient Boosting, and a regularized fully-connected neural network to provide a broad view of the performance landscape on this specific task.

C. Digital Twin Simulation for Traffic Management

The digital twin paradigm—maintaining a synchronized virtual replica of a physical system for monitoring, prediction, and control—has seen growing adoption in urban traffic management [1], [3], [4], [7]. Abdallah and Alghamdi [1] demonstrate a GRU-based digital twin for real-time traffic signal optimization, achieving a 33% improvement in predictive accuracy and a 15% reduction in CO₂ emissions. Their architecture couples IoT sensor feeds with a simulation model that validates control actions before applying them to the physical network—a design principle we adopt at the evaluation level by stress-testing classifiers within a simulated degraded sensor environment.

Bagabaldo and Hackl [4] review over 70 papers on digital twins for intelligent intersections, identifying three key benefits: testing interventions without real-world risk, generating synthetic training data, and detecting anomalies by comparing predicted vs. observed system states. Our noise injection module operationalizes the second benefit. Kaewunruen and Lian [7] describe the real-time synergy between live data streams and simulation models as the core digital twin architectural pattern, while Al-Rakhimi and Al-Mashari [3] identify traffic management as the highest-priority application domain for urban digital twin deployments.

D. Sensor Noise and Robustness in Traffic Monitoring

Real-world traffic sensor deployments are subject to multiple degradation mechanisms. Xie et al. [14] document that distributed acoustic sensing (DAS) systems are susceptible to polarization fading and quasi-

random optical noise, with accuracy dropping substantially under low signal-to-noise ratio conditions—motivating systematic robustness evaluation across all sensor modalities. Liu et al. [9] show that dropout regularization and noisy data augmentation improve generalization under deployment-time distribution shift, providing the closest methodological precedent to our noise injection protocol.

At the general ML level, Ibias et al. [6] develop a theoretical framework for noise robustness through input abstractions, demonstrating significantly less performance degradation under additive and dropout noise for models with abstraction layers. While their experiments focus on network traffic classification, the analytical framework applies directly to our sensor-count inputs, and their findings suggest that ensemble methods’ implicit averaging confers natural robustness—consistent with our experimental design.

V. PROPOSED METHODOLOGY

Our system is composed of four sequential stages: data preprocessing and feature engineering, chronological train/test splitting with feature scaling, sensor noise simulation, and multi-model training with paired evaluation.

A. Dataset

The dataset is a real-world traffic sensor log collected over two months at a monitored road segment, recorded at 15-minute intervals (96 samples per day, $\approx 5,952$ total records after cleaning). Each record contains: *Time*, *Date*, *Day of the week*, *CarCount*, *BikeCount*, *BusCount*, *TruckCount*, *Total*, and the target label *Traffic Situation* $\in \{\text{normal, low, high, heavy}\}$.

The class distribution is imbalanced: *normal* accounts for 3,610 records ($\approx 61\%$), *heavy* for 1,137 ($\approx 19\%$), *low* for 834 ($\approx 14\%$), and *high* for only 371 ($\approx 6\%$).

B. Data Preprocessing and Feature Engineering

Raw data is deduplicated and rows with missing values are dropped. The *Time* column is parsed in 12-hour format to extract the hour of day as an integer in $[0, 23]$. To capture the cyclic nature of time—where hour 23 is adjacent to hour 0—the hour is projected onto the unit circle:

$$\begin{aligned} \text{Hour_Sin} &= \sin\left(\frac{2\pi \cdot \text{Hour}}{24}\right), \\ \text{Hour_Cos} &= \cos\left(\frac{2\pi \cdot \text{Hour}}{24}\right) \end{aligned} \quad (2)$$

This two-dimensional representation ensures that the distance between adjacent hours is uniform across the midnight boundary, which a raw integer encoding would distort [8]. The *Day of the week* column is one-hot encoded into seven binary indicator variables (all seven days retained to avoid ambiguity in the multi-class context). The final feature vector is:

$$\mathbf{x} = [\text{HS}, \text{HC}, \text{CC}, \text{BC}, \text{BuC}, \text{TC}, \mathbf{d}_{\text{Mon}}, \dots, \mathbf{d}_{\text{Sun}}] \quad (3)$$

where HS = Hour_Sin, HC = Hour_Cos, CC = CarCount, BC = BikeCount, BuC = BusCount, TC = TruckCount. The *Total* column is excluded from the feature set to avoid a direct linear proxy for the

label. The target is integer-encoded via a `LabelEncoder`, mapping the four string classes to indices 0–3.

C. Chronological Train/Test Split and Feature Scaling

To respect the temporal ordering of the data and prevent any form of data leakage, the dataset is split chronologically at day boundaries. The total number of complete days is $n_{\text{days}} = \lfloor T/96 \rfloor$, and the split point is set such that the last 20% of days form the test set:

$$\text{split_day} = n_{\text{days}} - \max(1, \text{round}(0.2 \times n_{\text{days}})) \quad (4)$$

All rows before `split_day` constitute the training set; all remaining rows form the test set. Random shuffling is explicitly avoided. Feature standardization is applied using a `StandardScaler` fitted exclusively on the training set and then applied to both splits—preventing test-set statistics from influencing the scaling [11].

D. Sensor Noise Simulation

To emulate the degradation conditions encountered in real-world deployments, a noisy test set $\tilde{\mathcal{D}}_{\text{test}}$ is constructed using two independent perturbations applied via a seeded random number generator (seed=42 for reproducibility):

Occlusion noise. Each of the four vehicle count features (CarCount, BikeCount, BusCount, TruckCount) is perturbed by an additive integer drawn uniformly from $\{-3, \dots, +3\}$, then clipped at zero to prevent physically impossible negative values. This simulates the systematic undercounting that occurs when vehicles partially occlude one another in a camera’s field of view [9], [14].

Sensor dropout. For 2% of randomly selected records, one vehicle count column chosen uniformly at random is set to zero, simulating complete sensor loss for a single vehicle category—a failure mode observed in both loop detector malfunctions and camera feed interruptions [14].

Noise is applied *only* to the test set. The training set remains clean, reflecting the realistic scenario where models are trained on curated historical data but deployed against imperfect live feeds. Both $\mathcal{D}_{\text{test}}$ and $\tilde{\mathcal{D}}_{\text{test}}$ share identical labels y_{test} ; only the feature values differ.

E. Model Architectures and Hyperparameters

Five classifiers spanning three algorithmic families are trained on the same standardized training set and evaluated on both test conditions:

Logistic Regression (LR). SAGA solver, 2,000 maximum iterations, balanced class weights (random state=42). Serves as a linear lower-bound reference.

Random Forest (RF). 300 decision trees grown with bootstrap sampling, balanced class weights, full parallelization (random state=42). Averaging over many trees provides implicit variance reduction expected to translate into noise robustness [13].

Extra Trees (ET). 600 extremely randomized trees, balanced class weights (random state=42). Unlike RF, split thresholds are selected randomly rather than optimally, further reducing variance; the larger ensemble size (600 vs. 300 in RF) compensates for higher individual-tree error.

Histogram Gradient Boosting (HGB). Maximum tree depth of 6 (random state=42). Discretizes continuous features into histograms for efficient training. Class imbalance is handled implicitly through the boosting loss landscape, as this implementation does not support a `class_weight` parameter directly.

Neural Network (NN). A fully-connected feedforward network with the following architecture:

$$\begin{aligned} \text{Input} &\rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{BN} \rightarrow \text{Dropout}(0.3) \\ &\rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{BN} \rightarrow \text{Dense}(32, \text{ReLU}) \\ &\rightarrow \text{Dense}(K, \text{Softmax}) \end{aligned}$$

where $K = 4$ is the number of congestion classes. Compiled with the Adam optimizer and sparse categorical cross-entropy loss. Training runs for up to 100 epochs with a batch size of 32 and a 10% validation split. Early stopping monitors validation loss with a patience of 10 epochs and restores the best weights upon termination. The Dropout and Batch Normalization layers improve generalization under the noise-induced distribution shift between clean and noisy test sets [6].

F. Evaluation Protocol and Metrics

Each of the five models is evaluated on both $\mathcal{D}_{\text{test}}$ (clean) and $\tilde{\mathcal{D}}_{\text{test}}$ (noisy), producing two prediction vectors per model. The following metrics are computed for each (model, setting) pair:

- **Accuracy:** fraction of correctly classified samples.
- **Balanced Accuracy (BalAcc):** mean per-class recall, compensating for class imbalance.
- **F1-Macro:** unweighted mean of per-class F1 scores—the primary metric for imbalanced classification [8].
- **Precision-Macro / Recall-Macro:** unweighted per-class means.
- **Robustness Decay ($\Delta F1$):** as defined in Equation (1); lower is better.

Confusion matrices are generated for all 10 (model $\times$ setting) combinations and saved as heatmap visualizations using the `magma` colormap. A horizontal bar chart ranks all five models by $\Delta F1$ to provide an at-a-glance robustness comparison.

G. Artifact Saving

All outputs are written to `outputs/traffic_final_research/` for reproducibility. Table I lists every saved file.

VI. RESULTS AND DISCUSSION

This section will be completed after experiments are executed. Placeholder tables are provided below to define the reporting structure.

A. Classification Performance—Clean Setting

Table II will report all five metrics for each model evaluated on the clean test set $\mathcal{D}_{\text{test}}$.

TABLE I
SAVED ARTIFACTS AND FILENAMES

Artifact	Filename
Trained Keras neural network	traffic_nn_model.keras
Fitted StandardScaler	traffic_scaler.pkl
Fitted LabelEncoder	traffic_encoder.pkl
Confusion matrices (10 total)	cm_{model}_{setting}.png
Robustness decay chart	robustness_decay.png
Full metrics table	full_experiment_results.csv

TABLE II
CLASSIFICATION PERFORMANCE ON CLEAN TEST DATA

Model	Acc.	BalAcc	F1	Prec.	Rec.
Logistic Reg.	—	—	—	—	—
Random Forest	—	—	—	—	—
Extra Trees	—	—	—	—	—
Hist. GBM	—	—	—	—	—
Neural Network	—	—	—	—	—

B. Classification Performance—Noisy Setting

Table III will report the same five metrics for all models evaluated on the noisy test set $\tilde{\mathcal{D}}_{\text{test}}$, which applies occlusion (± 3 count perturbation) and sensor dropout (2% zero-out rate).

TABLE III
CLASSIFICATION PERFORMANCE ON NOISY TEST DATA

Model	Acc.	BalAcc	F1	Prec.	Rec.
Logistic Reg.	—	—	—	—	—
Random Forest	—	—	—	—	—
Extra Trees	—	—	—	—	—
Hist. GBM	—	—	—	—	—
Neural Network	—	—	—	—	—

C. Robustness Decay Analysis

Table IV will summarize the robustness decay ($\Delta F1$, Equation (1)) for each model, ranked from most to least robust. The accompanying bar chart generated by the pipeline will provide a visual ranking.

TABLE IV
ROBUSTNESS DECAY—F1-MACRO DROP (CLEAN $\rightarrow$ NOISY)

Model	F1 (Clean)	F1 (Noisy)	$\Delta F1 \downarrow$
Logistic Reg.	—	—	—
Random Forest	—	—	—
Extra Trees	—	—	—
Hist. GBM	—	—	—
Neural Network	—	—	—

D. Per-Class Analysis and Confusion Matrices

Confusion matrices for all 10 (model $\times$ setting) combinations will be analyzed here, with particular attention to the minority classes *high* and *low*, which are most vulnerable to noise-induced misclassification. The effect of class-weighted training (LR, RF, ET) on minority-class recall will be contrasted against the unweighted HGB model.

E. Discussion

This subsection will interpret the results in light of the related work reviewed in Section IV. Specifically, it will address: (i) whether the NN vs. GBDT performance gap reported by McElfresh et al. [11] holds under this traffic classification task; (ii) whether ensemble averaging translates to lower robustness decay as suggested by Ibias et al. [6]; and (iii) how our system compares to the GRU-based digital twin approach of Abdallah and Alghamdi [1] in terms of classification accuracy and operational reliability under sensor degradation.

VII. CONCLUSION AND FUTURE WORK

This paper presented a traffic congestion classification system grounded in real sensor data, evaluated under both clean and simulated noisy conditions to bridge the gap between benchmark performance and operational robustness. Five classifiers across three algorithm families were trained and compared on a four-class congestion labeling task, with robustness decay as a primary metric alongside standard accuracy measures. A digital twin-inspired noise injection layer simulates physically grounded sensor failure modes—camera occlusion and dropout—enabling a rigorous pre-deployment stress test absent from most existing traffic ML benchmarks.

Future work will extend this system in three directions: (1) integrating road-network topology to enable spatially-aware modeling using graph neural networks; (2) replacing static noise injection with a learned digital twin model that adapts its failure simulation to observed degradation statistics; and (3) deploying the best-performing classifier in a real-time edge inference system with continuous retraining as new sensor data arrives.

REFERENCES

- [1] W. Abdallah and M. Alghamdi, "Digital twin-enabled AI for sustainable traffic management: real-time urban mobility optimization in smart cities," *PeerJ Comput. Sci.*, vol. 12, p. e3574, 2026. doi: 10.7717/peerj-cs.3574
- [2] F. Ali, A. Ali, M. Imran, R. A. Naqvi, M. H. Siddiqi, and K.-S. Kwak, "Machine learning approach on traffic congestion monitoring system in Internet of Vehicles," *J. Inf. Process. Syst.*, vol. 17, no. 6, pp. 1207–1224, 2021. doi: 10.1016/j.procs.2020.09.212
- [3] M. S. Al-Rakhami and M. Al-Mashari, "Digital twin technology in smart cities: A step toward intelligent urban management," *Energy Rep.*, vol. 13, pp. 1–15, 2025. doi: 10.1016/j.egy.2025.01.012
- [4] A. R. Bagabaldo and J. Hackl, "Digital twins for intelligent intersections: A literature review," *arXiv preprint arXiv:2510.05374*, 2025. doi: 10.48550/arXiv.2510.05374
- [5] L. Chen, Y. Zhang, and X. Wang, "Traffic congestion recognition based on convolutional neural networks in different scenarios," *Eng. Appl. Artif. Intell.*, vol. 145, p. 109782, 2025. doi: 10.1016/j.engappai.2025.109782
- [6] A. Ibias, K. Capala, V. Varma, A. Drózd, and J. L. R. Sousa, "Improving noise robustness through abstractions and its impact on machine learning," *arXiv preprint arXiv:2406.08428*, 2024. doi: 10.48550/arXiv.2406.08428
- [7] S. Kaewunruen and Q. Lian, "A digital twin in transportation: Real-time synergy of traffic data streams and simulation for virtualizing motorway dynamics," *Adv. Eng. Inform.*, vol. 55, p. 101858, 2022. doi: 10.1016/j.aei.2022.101858
- [8] A. A. Kashyap, R. Patil, T. J. Rama Sushma, A. Bhandari, R. Gupta, and A. Bhardwaj, "A survey on traffic flow prediction and classification," *Intell. Syst. Appl.*, vol. 18, p. 200205, 2023. doi: 10.1016/j.iswa.2023.200205
- [9] X. Liu, R. Chen, and J. Park, "Enhancing traffic monitoring with noise-robust distributed acoustic sensing and deep learning," *Transp. Res. C, Emerg. Technol.*, vol. 165, p. 104756, 2024. doi: 10.1016/j.trc.2024.104756
- [10] M. Mahmoud, A. Alkhatib, and H. Hamdoun, "Traffic congestion prediction using machine learning," in *Proc. IEEE Int. Conf. Comput. Inf. (ICCI)*, 2024. doi: 10.1109/ICCI.2024.10498418
- [11] D. C. McElfresh, S. Khandagale, J. Valverde, C. VishakPrasad, B. Feuer, C. Hegde, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023. doi: 10.48550/arXiv.2305.02997
- [12] R. Sharma, M. Gupta, and K. Patel, "Intelligent traffic flow prediction using deep learning techniques: A comparative study," *SN Comput. Sci.*, vol. 5, no. 4, p. 512, 2024. doi: 10.1007/s42979-024-03552-3
- [13] T. J. Ting, X. Li, S. Sanner, and B. Abdulhai, "Revisiting random forests in a comparative evaluation of graph convolutional neural network variants for traffic prediction," in *Proc. IEEE Int. Conf. Intell. Transp. Syst. (ITSC)*, 2021, pp. 1–8. doi: 10.1109/ITSC48978.2021.9564595
- [14] D. Xie, X. Wu, Z. Guo, H. Hong, B. Wang, and Y. Rong, "Intelligent traffic monitoring with distributed acoustic sensing," *Seismol. Res. Lett.*, 2024. doi: 10.1785/0220240298
- [15] Y. Zhang, H. Liu, Z. Wang, and J. Chen, "AI-powered urban transportation digital twin: Methods and applications," *arXiv preprint arXiv:2501.10396*, 2025. <https://arxiv.org/abs/2501.10396>